

回归学习

主要内容

- 回归学习的概念
- 线性回归学习
- 回归学习与二分类
 - 逻辑回归
 - 逻辑回归的代价函数
- 回归学习的正则化
- 回归学习与多分类
 - Softmax回归
 - Softmax回归的代价函数

➤ 回归学习

➤ 有监督学习的一种方法。该方法的**核心思想**是从连续型统计数据中得到数学模型，然后将该数学模型用于**预测**或者**分类**。

➤ 该方法处理的数据可以是多维的。

➤ 注释：回归是由达尔文的表兄弟Francis Galton发明的。Galton于1877年完成了第一次回归预测，目的是根据上一代豌豆的种子（双亲）的尺寸来预测下一代豌豆种子（孩子）的尺寸（身高）。Galton在大量对象上应用了回归分析，甚至包括人的身高。他得到的结论是：如果双亲的高度比平均高度高，他们的子女也倾向于平均身高但尚不及双亲，这里就可以表述为：孩子的身高向着平均身高回归。Galton在多项研究上都注意到了这一点，并将此研究方法称为回归。

问题引入

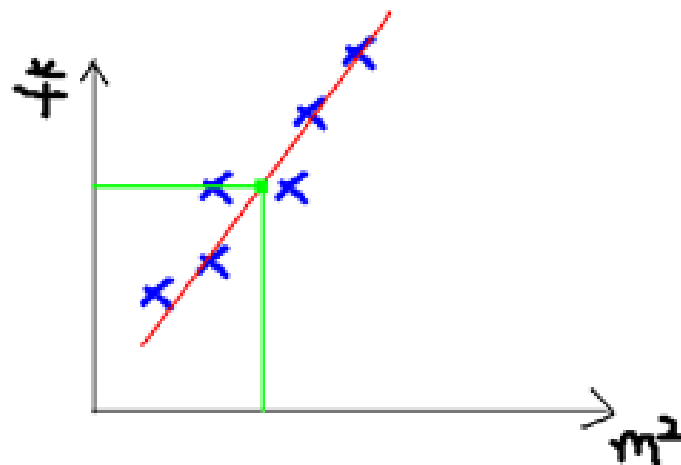
假设有一个房屋销售的数据如下：

面积(m^2)	销售价钱 (万元)
123	250
150	320
87	160
102	220



如果来了一个新的面积，假设在销售价钱的记录中没有的，怎么处理？

解决方法：用一条曲线去尽量准的拟合这些数据，之后如果有新的输入过来，我们可以在将曲线上这个点对应的值返回。如果用一条直线去拟合，可能是下面的样子：



将问题描述为机器学习问题：



房屋记录表：

- 一条房屋数据为一个训练实例，记为 x
- 所有房屋数据构成训练集(training set)



房屋销售价钱：

- 目标数据值，与实例对应，一般记为 y



拟合的函数（或者称为假设或者模型）：

- 一般写做 $y = h(x)$

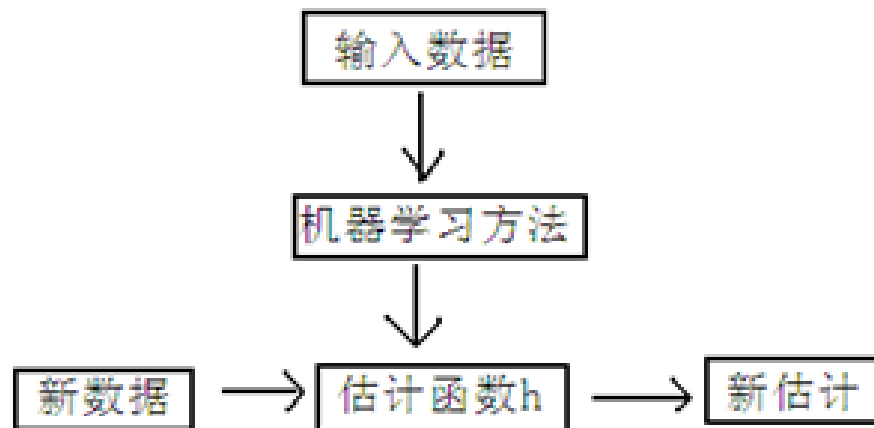


数据特征：

- 一个训练实例由若干特征描述，特征的数目也是实例的维度
- 本例的特征是一维，目标值也是一维。回归方法能够解决特征多维，目标值是多维度离散值或连续值的问题。

学习过程

首先给出输入数据(训练数据)，算法通过一系列的过程得到一个估计的函数，这个函数有能力对没有见过的新数据给出一个新的估计，也被称为构建一个模型。就如同上面的线性回归函数。



一个典型的回归学习的过程

- 线性回归 (Linear regression)
 - 假设回归模型为线性函数，利用代价函数对自变量和因变量之间关系进行建模的一种回归分析.
 - 线性回归属于**有监督学习**，因此线性回归和有监督学习是相似的，先给定一个训练集，根据这个训练集学习出一个**线性函数**，之后测试这个函数的训练精度（即此函数是否足够拟合训练集数据），挑选出最好的函数（cost function最小）即可.

回归学习-线性回归

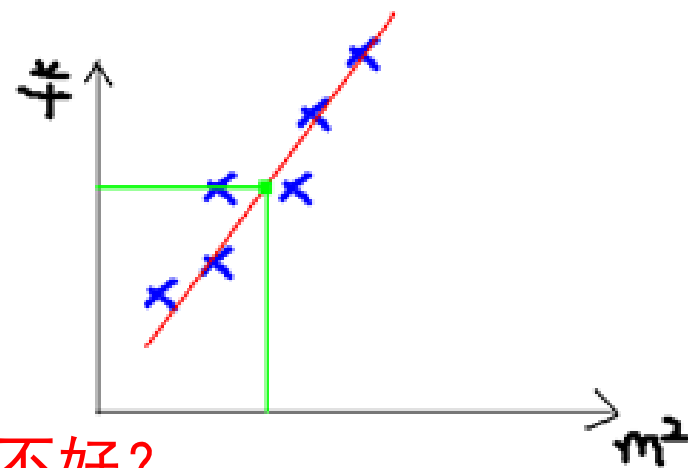
构建回归模型：

- (1) 因为是线性回归，事先假定要学习的函数为**线性函数**，即直线函数
- (2) 本例是单变量，因此只有一维特征 x ；

单变量线性回归模型

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

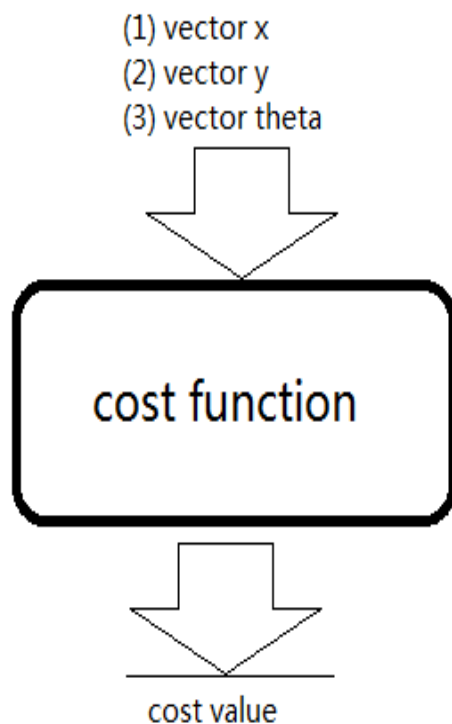
x : feature, $h(x)$: hypothesis;



问题：如何评价线性函数拟合的好坏？

设计代价函数：

代价函数 (Cost Function)：对假设进行评价，cost function 的值越小，说明与训练数据拟合的越好；



解释：

比如存在数据集(1,1)、(2,2)、(3,3)，则 $x=[1;2;3]$ ， $y=[1;2;3]$

如果假设预测了函数为 $y = x$ ，则发现cost value = 0；

如果假设预测了函数为 $y = 2x$ ，则发现cost value = 1；

如果假设预测了函数为 $y = 3x$ ，则发现cost value = 2；

则我们发现 $y = x$ 的cost value最小，因此选择 $y=x$ 作为我们的 hypothesis

回归学习-线性回归

定义代价函数 (Cost Function) :

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

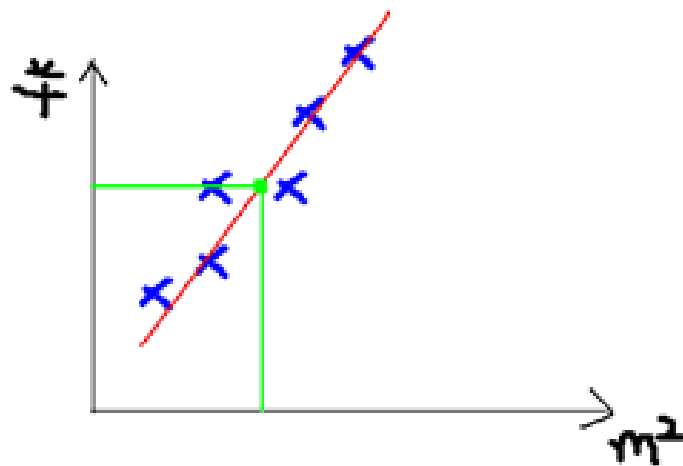
其中:

$x^{(i)}$ 表示训练集中的第*i*个训练实例;

$y^{(i)}$ 表示第*i*个训练实例的目标值;

$h_{\theta}(x^{(i)})$ 表示假设函数;

m 为训练样例的数量;



例: 给定数据集 (1, 1)、(2, 2)、(3, 3)

如果我们预测 $\theta_0 = 0$, $\theta_1 = 1$, 则 $h(x) = x$, 则 cost function:

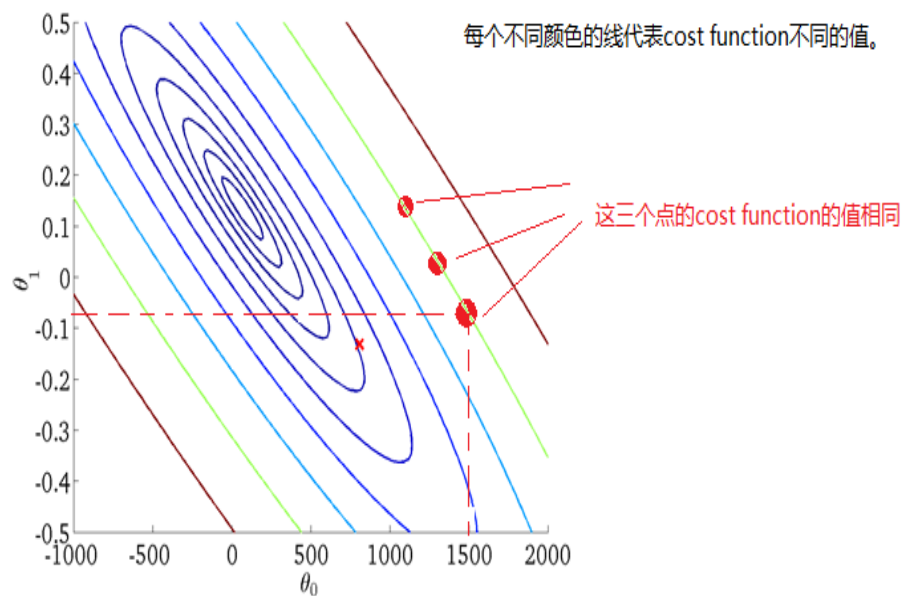
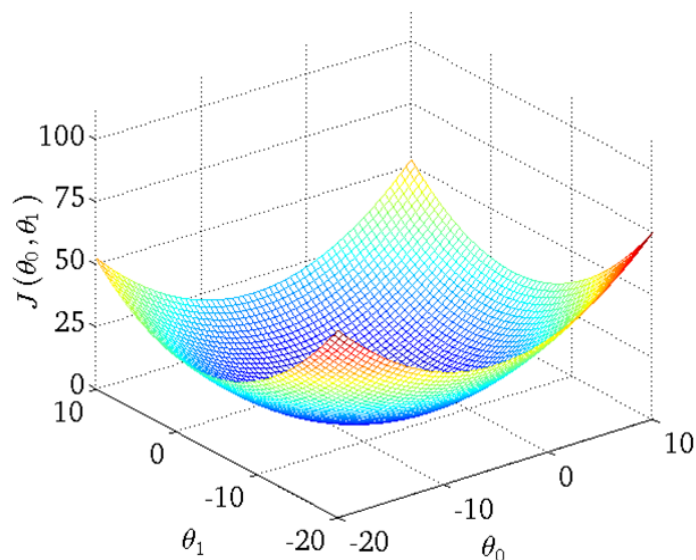
$$J(0, 1) = 1/(2*3) * [(h(1)-1)^2 + (h(2)-2)^2 + (h(3)-3)^2] = 0;$$

如果我们预测 $\theta_0 = 0$, $\theta_1 = 0.5$, 则 $h(x) = 0.5x$, 则 cost function:

$$J(0, 0.5) = 1/(2*3) * [(h(1)-1)^2 + (h(2)-2)^2 + (h(3)-3)^2] = 0.58;$$

分析代价函数与参数的关系：

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



注意：如果是线性回归，则cost function J 与 θ_0 与 θ_1 的函数一定是碗状的，即只有一个最小点；

模型参数求解：Gradient Descent（梯度下降）

找出cost function函数的最小值；

梯度下降原理：将函数比作一座山，站在某个山坡上，往四周看，从哪个方向向下走一小步，能够下降的最快；

方法：

(1) 先确定向下一步的步伐大小，我们称为Learning rate；

(2) 任意给定一个初始值：

θ_0 θ_1

(3) 确定一个向下的方向，并向下降预先规定的步伐，并更新

θ_0 θ_1

(4) 当下降的高度小于某个定义的值，则停止下降；

如何确定向下的方向:

线性回归需要求出 θ_0 θ_1 , 使得cost function的最小;
 因此我们计算cost function梯度, 梯度反方向是代价函数值下降最快的方向, 将梯度下降和线性回归进行整合, 学习规则如下所示:

Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
 }

Linear Regression Model

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

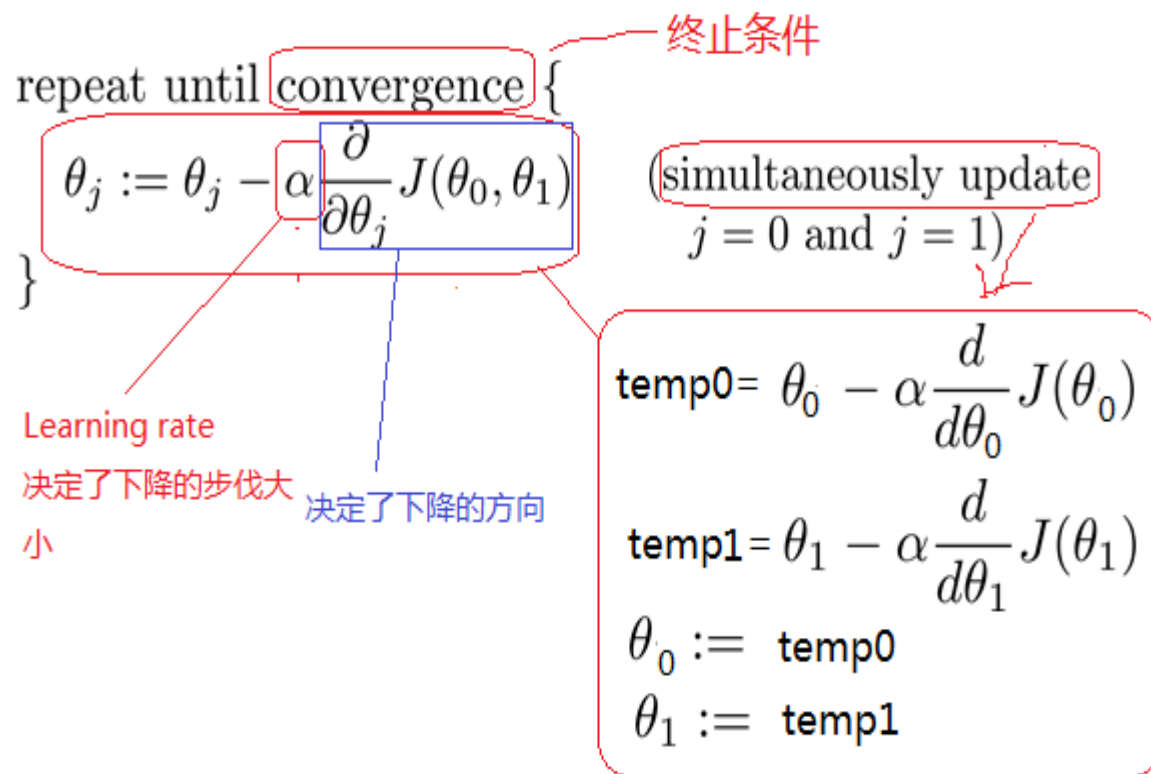
repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

}

梯度下降算法分析：



特点：

- (1) 模型为线性模型，代价函数不存在局部极小值，只有全局最小值；
- (2) 越接近最小值时，下降的步伐要越小；

梯度下降算法分析：假设代价函数是复杂非线性函数

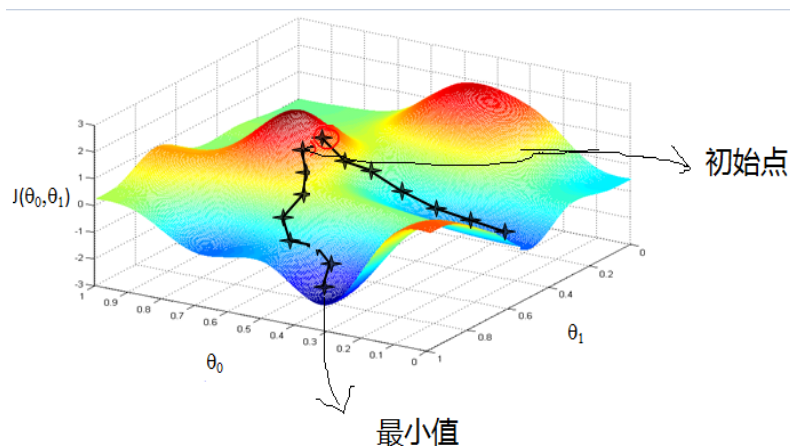
问题：

如果 θ_0, θ_1 初始值就在 local minimum 的位置，则 θ_0, θ_1 会如何变化？

答：因为 θ_0, θ_1 已经在 local minimum 位置，所以代价函数对参数的导为 0，因此 θ_0, θ_1 不会变化；

问题：正确的 α ，应使代价函数越来越小，如何选取 α ？

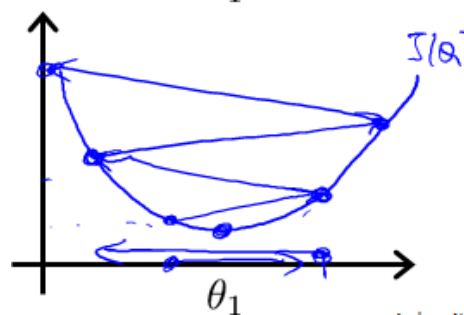
答：随时观察 α 值，如果 cost function 变小了，则不变，反之，则再取一个更小的值；



从上图可以看出：**初始点不同，获得的极小值也不同**，因此梯度下降求得的只是**局部极小值**；

梯度下降算法分析：

注意：下降的步伐大小 α 非常重要，因为如果太小，则找到函数最小值的速度就很慢，如果太大，则可能会出现overshoot the minimum的现象；



overshoot minimum现象

如果 α 取某值后，发现 J 函数增大，则需要减小 α 的值；

线性回归求解：

➤ 最小二乘法

$$h_{\theta}(x) = \sum_j \theta_j x_j = \theta^{\top} x$$

➤ 可以直接用数学求解公式，但它要求X是列满秩

$$\theta = (X^T X)^{-1} X^T \vec{y}.$$

➤ 梯度下降法

$$J(\theta) = \frac{1}{2} \sum_i \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 = \frac{1}{2} \sum_i \left(\theta^{\top} x^{(i)} - y^{(i)} \right)^2$$

损失函数

$$\frac{\partial J(\theta)}{\partial \theta_j} = \sum_i x_j^{(i)} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \quad x^{(i)} \in R^n$$

二分类问题

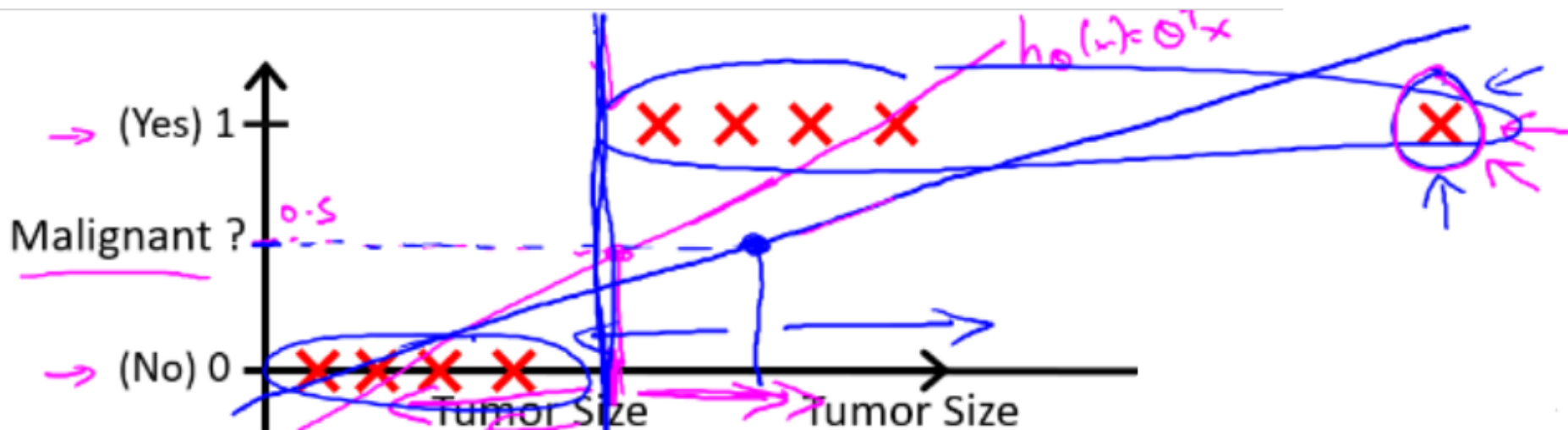
- 二分类问题是指预测的 y 值只有两个取值（0或1）。
 - 例如：垃圾邮件过滤系统，预测邮件的类别，垃圾邮件还是正常邮件。
 - 类别通常称为正类（positive class）和负类（negative class）
 - 垃圾邮件的例子中，正类就是正常邮件，负类就是垃圾邮件。
- 应用举例：是否垃圾邮件分类？是否肿瘤、癌症诊断？是否金融欺诈？

回归学习-逻辑回归

- 线性回归与二分类：

如果我们忽略二分类问题中 y 的取值是一个离散的取值（0或1），我们继续使用线性回归来预测 y 的取值，会导致什么问题：

- y 的取值并不为0或1，很难解释。
- 线性回归通过最小化均方误差（MSE）来拟合数据，离群点会显著影响直线拟合的方向，导致模型较大偏差。



回归学习-逻辑回归

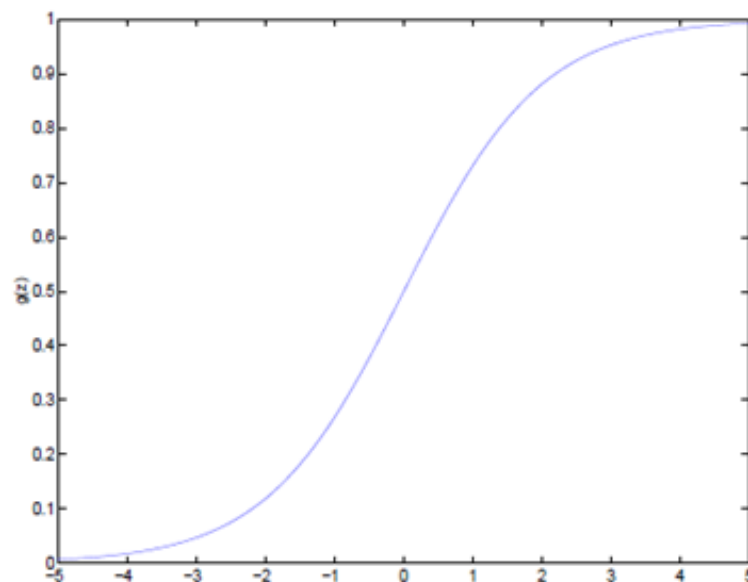
- 逻辑回归如何求解二分类问题：

逻辑回归使用一个函数来归一化y值，使y的取值在区间(0, 1)内，可看作时分类概率值，这个函数称为**Logistic函数**，也称为**Sigmoid函数**。函数公式如下：

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$Z = \sum_j \theta_j x_j = \theta^\top x$$

- Logistic函数当z趋近于无穷大时， $g(z)$ 趋近于1；当z趋近于无穷小时， $g(z)$ 趋近于0。Logistic函数的图形如图：



逻辑回归 (Logistic Regression) 讨论:

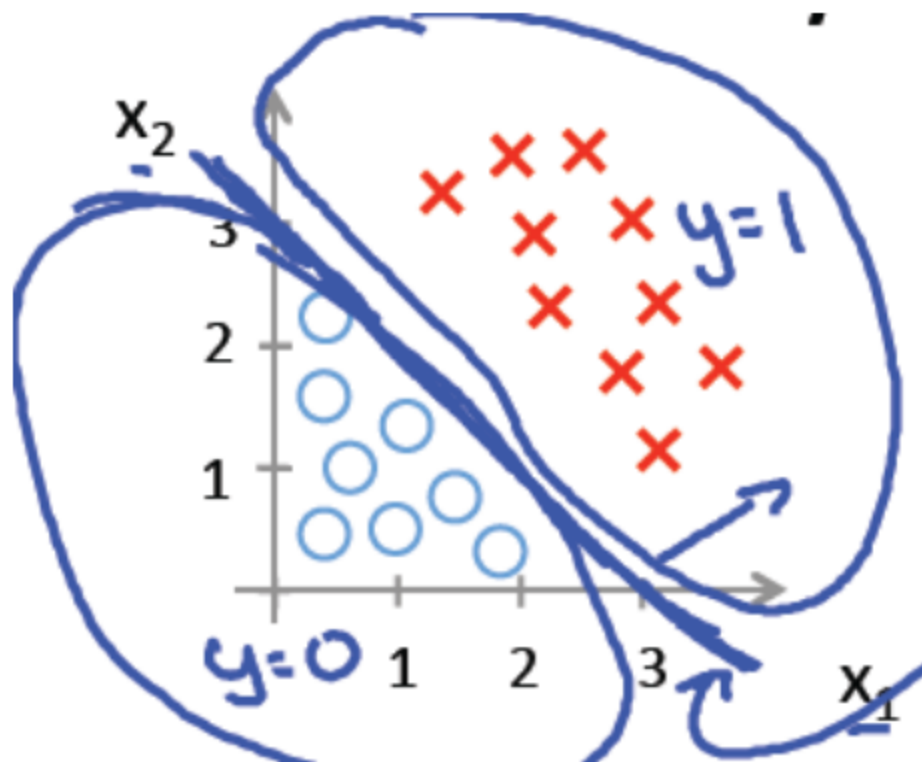
- 逻辑回归的模型本质上是一个线性回归模型
 - 除去sigmoid映射函数关系, 其他的步骤, 算法都是线性回归计算
 - 可以说, 逻辑回归, 是以线性回归为理论支持
- 相比线性模型, Sigmoid函数对离群点带来的偏差有一定的抑制作用。
- sigmoid函数的输出可以理解为 $h(x)=1$ 的概率
- sigmoid 函数的求导特点
- 逻辑回归可引入高阶项, 实现非线性分类

Sigmoid函数(sigmoid function)的性质:

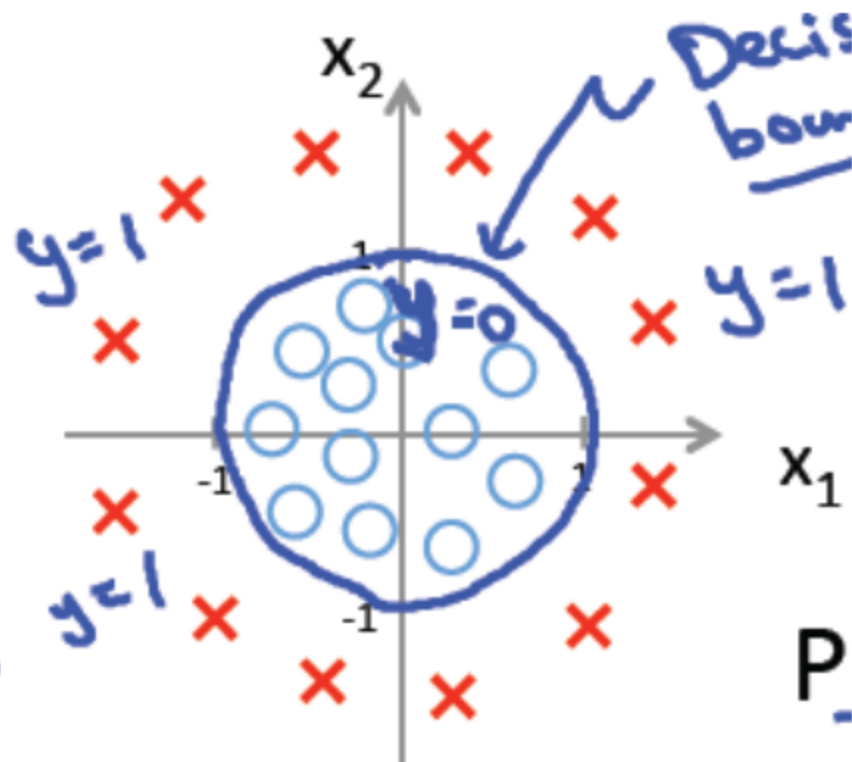
$$\begin{aligned}g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\&= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\&= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})}\right) \\&= g(z)(1 - g(z)).\end{aligned}$$

回归学习- 逻辑回归

传统逻辑回归只能分类线性可分数据，若引入高阶项，
如： $z = x_1^2 + x_2^2 - 1$ ，可实现非线性分类



线性决策边界



非线性决策边界

回归学习-逻辑回归

逻辑回归与概率输出：

对于线性边界的情况，边界形式如下：

$$\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \sum_{i=1}^n \theta_i x_i = \theta^T x$$

构造预测函数为：

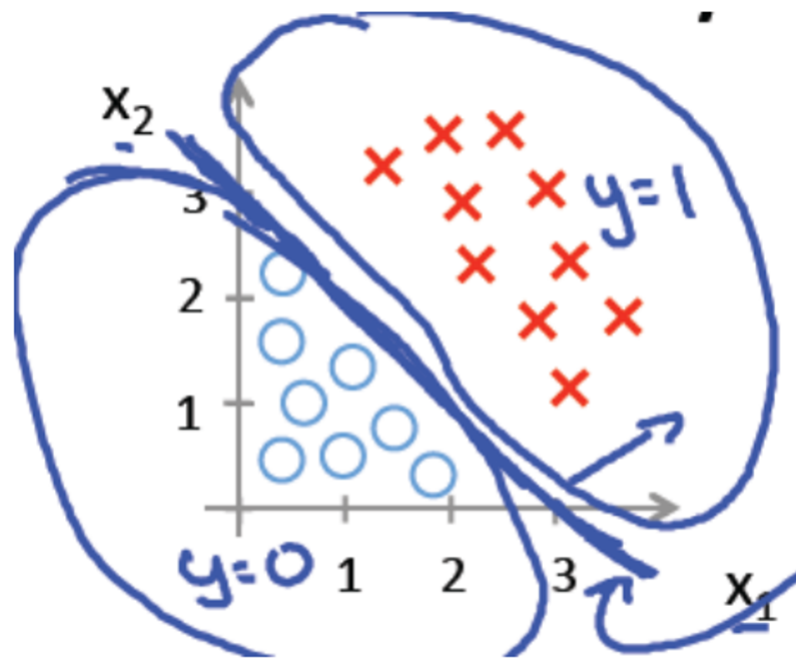
$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

函数 $h_{\theta}(x)$ 的值有特殊的含义，它表示结果

取1的概率，因此对于输入x，其分类结果为

类别1和类别0的概率分别为：

$$\begin{aligned} P(y=1 | x; \theta) &= h_{\theta}(x) \\ P(y=0 | x; \theta) &= 1 - h_{\theta}(x) \end{aligned} \quad (1)$$



线性决策边界

逻辑回归的损失函数J:

损失函数是基于最大似然估计推导得到的(见Bayes章中的证明), 称为交叉熵损失函数。

Logistic regression cost function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= \underbrace{-\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]}_{\uparrow} \end{aligned}$$

逻辑回归交叉熵损失函数推导

– $D = \{ \langle x_1, d_1 \rangle, \dots, \langle x_m, d_m \rangle \}$

– $P(D | h) = \prod_{i=1}^m P(x_i, d_i | h) = \prod_{i=1}^m P(d_i | h, x_i) P(x_i | h) = \prod_{i=1}^m P(d_i | h, x_i) P(x_i)$

– 注意:

$$P(d_i | h, x_i) = \begin{cases} h(x_i) & d_i = 1 \\ 1 - h(x_i) & d_i = 0 \end{cases} = h(x_i)^{d_i} (1 - h(x_i))^{1-d_i}$$

$$P(D | h) = \prod_{i=1}^m h(x_i)^{d_i} (1 - h(x_i))^{1-d_i} P(x_i)$$

$$h_{ML} = \arg \max_{h \in H} \prod_{i=1}^m h(x_i)^{d_i} (1 - h(x_i))^{1-d_i} p(x_i)$$

$$= \arg \max_{h \in H} \prod_{i=1}^m h(x_i)^{d_i} (1 - h(x_i))^{1-d_i}$$

$$= \arg \max_{h \in H} \sum_{i=1}^m d_i \ln h(x_i) + (1 - d_i) \ln(1 - h(x_i)) \quad (6.13)$$

回归学习-逻辑回归

基于梯度下降算法的参数更新规则：

To fit parameters θ :

$$\min_{\theta} J(\theta) \quad \text{Get } \underline{\theta}$$

To make a prediction given new $\underline{x}$:

Output $\underline{h_{\theta}(x)} = \frac{1}{1+e^{-\theta^T x}}$

$$\underline{p(y=1 | x; \theta)}$$

$$\frac{\partial(J(\theta))}{\partial \theta_j}$$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

θ 更新公式

梯度下降学习规则推导：

$$\frac{\partial J}{\partial w_j} = -\frac{1}{N} \sum_{i=1}^N \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial z_i} \frac{\partial z_i}{\partial w_j}$$

with

$$J = -\frac{1}{N} \sum_{i=1}^N y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)$$

and

$$p_i = \frac{1}{1 + e^{-z_i}}$$

and

$$z_i = X_i w^T + b$$

with $X = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,n} \\ x_{2,1} & x_{2,2} & \dots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,1} & x_{N,2} & \dots & x_{N,n} \end{bmatrix}$, where n is representing the number of

independent variables and N the number of samples,
the weight vector $w = [w_0 \ \dots \ w_{n-1}]$ and a scalar b representing the bias term.

回归学习-逻辑回归

梯度下降学习规则推导：

$$\begin{aligned}
 \frac{\partial J}{\partial w_j} &= -\frac{1}{N} \sum_{i=1}^N \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial z_i} \frac{\partial z_i}{\partial w_j} \\
 &= -\frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i} (-1) \right] [p_i(1-p_i)] x_j = \\
 &= -\frac{1}{N} \sum_{i=1}^N [y_i(1-p_i) - (1-y_i)p_i] x_j = \\
 &= -\frac{1}{N} \sum_{i=1}^N (y_i - p_i) x_j = \\
 &= \frac{1}{N} \sum_{i=1}^N (p_i - y_i) x_j
 \end{aligned}$$

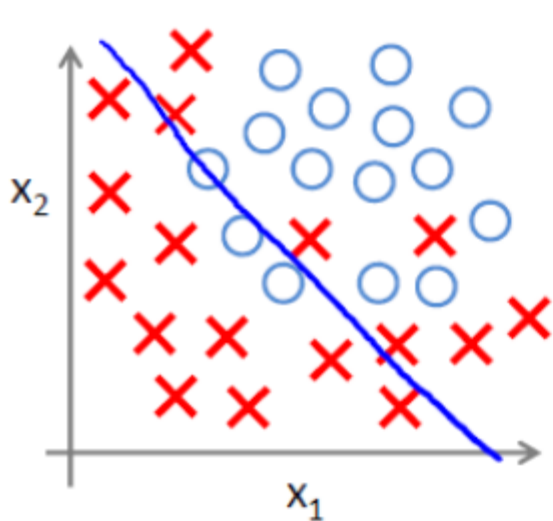
The partial derivative of the cost function J with respect to the bias b can be calculated accordingly. Considering that the mathematical derivation of the formula is very similar, except that $\frac{\partial z_i}{\partial b} = 1$, we can simply write:

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)$$

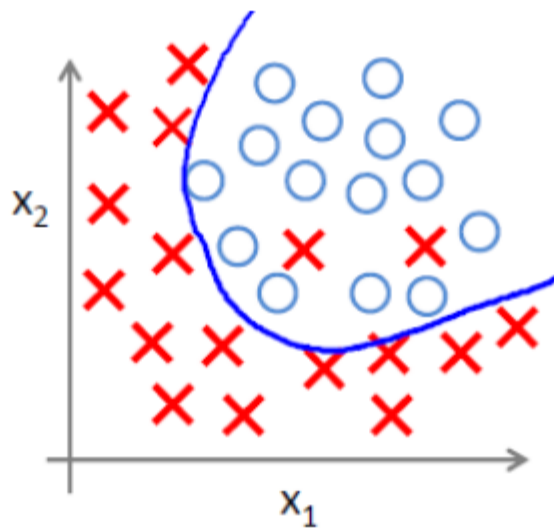
回归学习-正则化Regularization

过拟合问题

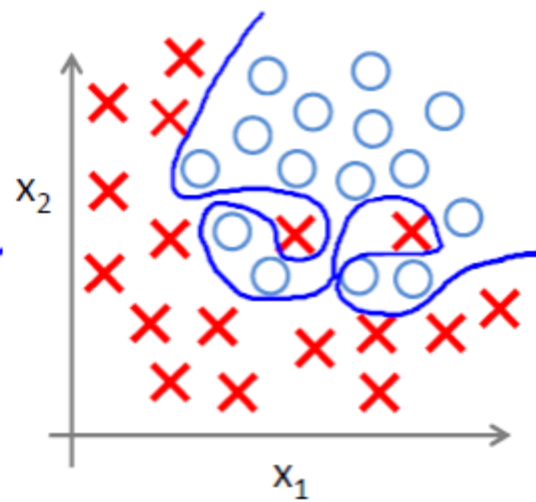
逻辑回归也存在过拟合的风险，特别是当引入高阶项时，模型的复杂度变大，具有较强的非线性拟合能力，容易导致过拟合，使得模型的泛化能力较差（对未知数据的预测能力）。



欠拟合



合适拟合



过拟合

回归学习：正则化Regularization

➤ 问题的主因：

过拟合问题源自过多的特征，以及高度复杂的模型

➤ 解决方法

1) 减少特征数量（减少特征会失去一些信息，即使特征选的很好）

- 人工选择要保留的特征；
- 模型选择算法：一般采用交叉验证，选择性能最好的模型；

2) 正则化（特征较多时比较有效）：保留所有特征，但减小结构风险

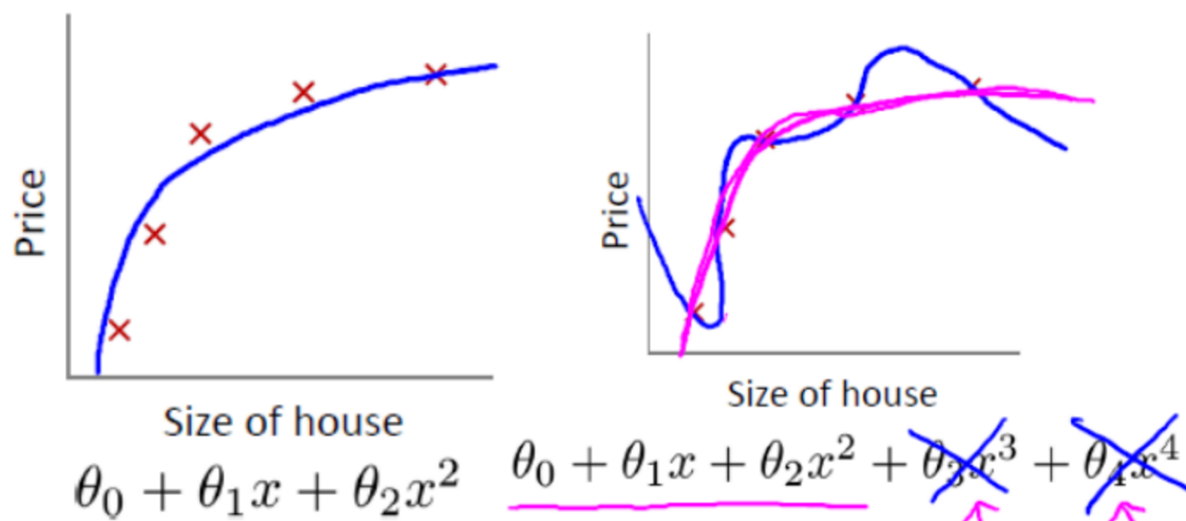
- 经验风险：机器学习模型关于训练数据集的平均损失称为经验风险
- 结构风险：结构风险是在经验风险的基础上加上表示模型复杂度的正则项（惩罚项）

回归学习-正则化Regularization

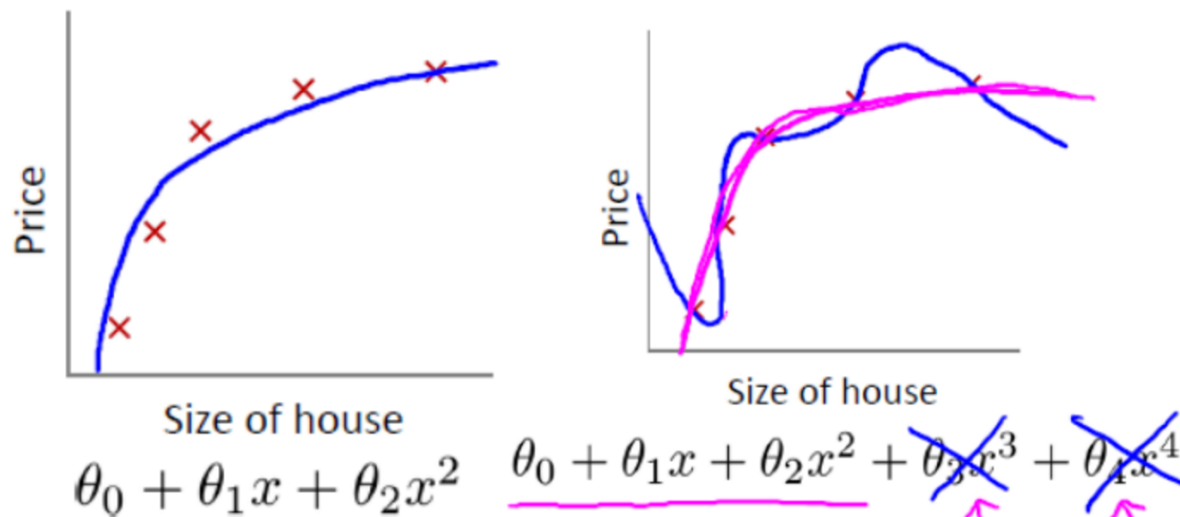
正则化方法

正则化是结构风险最小化策略的实现，是在经验风险上加一个正则化项或惩罚项。正则化项一般是模型复杂度的单调递增函数，模型越复杂，正则化项就越大。

房价预测问题，多项式回归



正则化Regularization



直观来看，如果我们想解决这个例子中的过拟合问题，最好能将 x^3, x^4 的影响消除，也就是让 $\theta_3 \approx 0, \theta_4 \approx 0$ 。假设我们对 θ_3, θ_4 进行惩罚，并且令其很小，一个简单的办法就是给原有的Cost函数加上两个略大惩罚项，例如：

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^n (h_{\theta}(x_i) - y_i)^2 + 1000\theta_3^2 + 1000\theta_4^2$$

结构风险

这样在最小化Cost函数的时候， $\theta_3 \approx 0, \theta_4 \approx 0$ 。经验风险

正则化Regularization

正则项可以取不同的形式，在回归问题中取平方损失，就是参数的L2范数，也可以取L1范数。取平方损失时，模型的损失函数变为：

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^n (h_{\theta}(x_i) - y_i)^2 + \lambda \sum_{j=1}^n \theta_j^2$$

λ 是正则项系数：

- 如果它的值很大，说明对模型的复杂度惩罚大，对拟合数据的损失惩罚小，这样它就不会过分拟合数据，在训练数据上的偏差较大，但是可能出现欠拟合的现象；
- 如果它的值很小，说明比较注重对训练数据的拟合，在训练数据上的偏差会小，但是可能会导致过拟合。

多分类问题：假设预测值 y 有 k 种可能，即 $y \in \{1, 2, \dots, k\}$

比如 $k=3$ 时，可以看作是要将一封未知邮件分为垃圾邮件、个人邮件还是工作邮件这三类。如何实现？

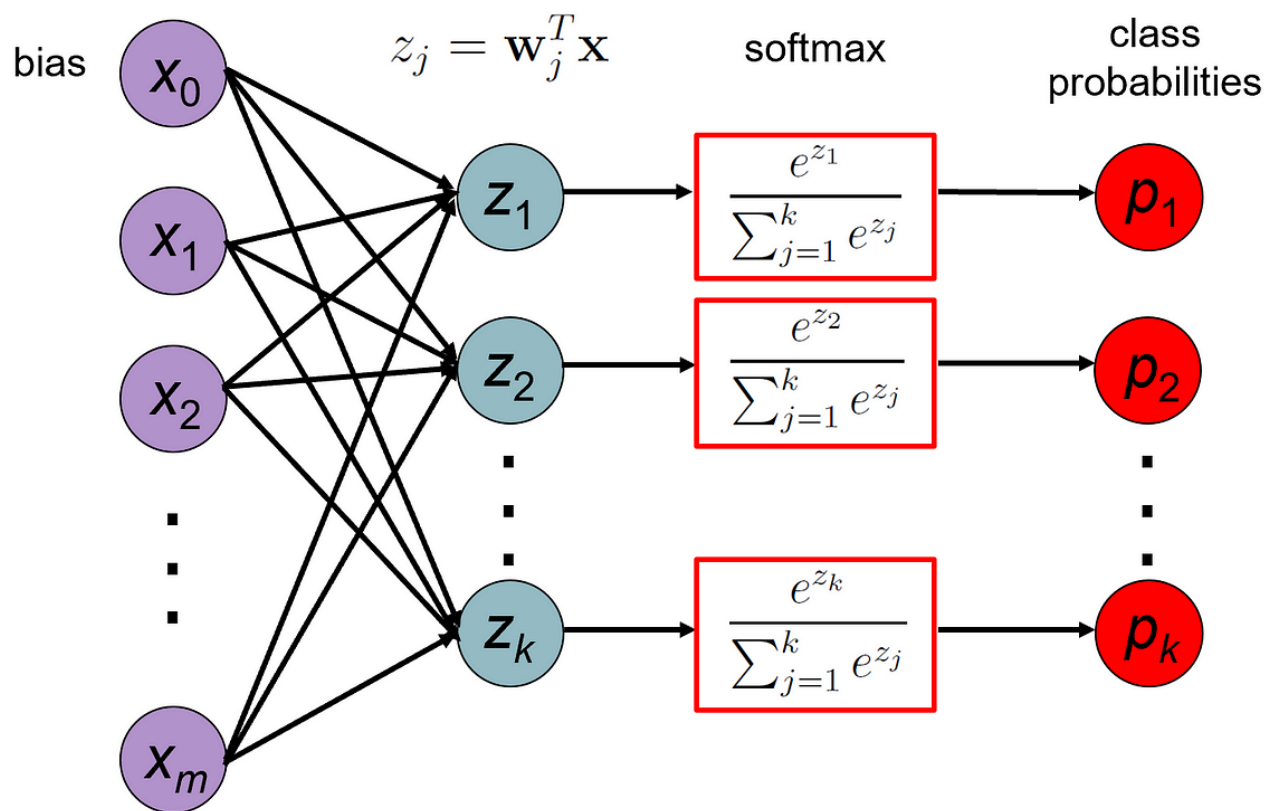
可以采用多个线性函数，并对其输出进行归一化，转为概率

$$\begin{aligned}\theta_1 &= x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1, \\ \theta_2 &= x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2, \\ \theta_3 &= x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3.\end{aligned}$$

$$\begin{aligned}p(y = i | x; \theta) &= \frac{e^{\eta_i}}{\sum_{j=1}^k e^{\eta_j}} \\ &= \frac{e^{\theta_i^T x}}{\sum_{j=1}^k e^{\theta_j^T x}}\end{aligned} \quad h_{\theta}(x^{(i)}) = \begin{bmatrix} p(y^{(i)} = 1 | x^{(i)}; \theta) \\ p(y^{(i)} = 2 | x^{(i)}; \theta) \\ \vdots \\ p(y^{(i)} = k | x^{(i)}; \theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \begin{bmatrix} e^{\theta_1^T x^{(i)}} \\ e^{\theta_2^T x^{(i)}} \\ \vdots \\ e^{\theta_k^T x^{(i)}} \end{bmatrix}$$

回归学习-Softmax回归

Softmax回归的神经网络表示：



回归学习-Softmax回归

Softmax回归的损失函数的极大似然估计推导:

$$P(y^{(i)}|x^{(i)}) = P(y = y^{(i)}|x^{(i)}) \qquad \mathbf{1}(y = i) = \begin{cases} 1 & \text{if } y = i \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

$$\text{If } y^{(i)} = 1, \quad P(y^{(i)}|x^{(i)}) = P(y = 1|x^{(i)})$$

$$\text{If } y^{(i)} = 2, \quad P(y^{(i)}|x^{(i)}) = P(y = 2|x^{(i)})$$

$$\text{If } y^{(i)} = l, \quad P(y^{(i)}|x^{(i)}) = P(y = l|x^{(i)}) \quad \text{where } l \text{ can be either } 1, 2, 3, \dots, k$$

$$\text{So more compactly,} \quad P(y^{(i)}|x^{(i)}) = \prod_{l=1}^k \left(P(y^{(i)} = l|x^{(i)}) \right)^{\mathbf{1}(y^{(i)}=l)}$$

$$L(\theta) = P(y^{(1)}|x^{(1)}) \cdot P(y^{(2)}|x^{(2)}) \cdot \dots \cdot P(y^{(m)}|x^{(m)}) \quad \text{where } m \text{ is the total number of training data}$$

$$= \prod_{i=1}^m P(y^{(i)}|x^{(i)})$$

$$= \prod_{i=1}^m \prod_{l=1}^k \left(P(y^{(i)} = l|x^{(i)}) \right)^{\mathbf{1}(y^{(i)}=l)}$$

$$= \prod_{i=1}^m \prod_{l=1}^k \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)}$$

回归学习-Softmax回归

Softmax回归的损失函数的极大似然估计推导：

$$\ell(\theta) = \ln L(\theta)$$

$$= \ln \prod_{i=1}^m \prod_{l=1}^k \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)}$$

$$= \sum_{i=1}^m \ln \prod_{l=1}^k \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)}$$

$$= \sum_{i=1}^m \sum_{l=1}^k \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)}$$

$$\begin{aligned} -\ell(\theta) &= - \sum_{i=1}^m \sum_{l=1}^k \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)} \\ &= - \sum_{i=1}^m \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right) \end{aligned}$$

$$\max_{\theta} \ell(\theta) = - \min_{\theta} -\ell(\theta)$$

回归学习-Softmax回归

Softmax回归的损失函数如下：

$$\begin{aligned} -\ell(\theta) &= -\sum_{i=1}^m \sum_{l=1}^k \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)} \\ &= -\sum_{i=1}^m \sum_{l=1}^k \mathbf{1}(y^{(i)} = l) \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right) \end{aligned}$$

Softmax回归的参数学习规则如下，k为类别数目，n为样本的特征数目：

$$\theta_{gh} := \theta_{gh} - \alpha \sum_{i=1}^m \left(\frac{e^{\theta_g^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} - \mathbf{1}(y^{(i)} = g) \right) x_h^{(i)} \quad \text{where } \alpha \text{ is the learning rate}$$

where $g = 1, 2, \dots, k$ and $h = 0, 1, 2, \dots, n$

回归学习-Softmax回归

Softmax回归的损失函数也可写成如下形式：实际为交叉熵函数

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{j=1}^k 1 \{y^{(i)} = j\} \log \frac{e^{\theta_j^T x^{(i)}}}{\sum_{l=1}^k e^{\theta_l^T x^{(i)}}} \right] + \frac{\lambda}{2} \sum_{i=1}^k \sum_{j=0}^n \theta_{ij}^2$$

针对第*i*个样本的交叉熵损失：假设该样本属于第 $y^{(i)}$ 类

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^k y_j \log \hat{y}_j = - \sum_{j=1}^k 1 \{y^{(i)} = j\} \log \frac{e^{\theta_j^T x^{(i)}}}{\sum_{l=1}^k e^{\theta_l^T x^{(i)}}}$$

$$\mathbf{y} = (y_1, y_2, \dots, y_j, \dots) = (0, 0, \dots, 1, \dots)$$

回归学习-Softmax回归

Softmax回归的梯度下降推导：

$$\begin{aligned}
 l(\mathbf{y}, \hat{\mathbf{y}}) &= - \sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\
 &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\
 &= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j.
 \end{aligned}$$

q 为类别个数,
 o_j 为输出神经元 j 的净输入

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j.$$

$$\mathbf{o}_j = \mathbf{w}_j^T \mathbf{x}$$

$$\mathbf{w}_j = \mathbf{w}_j - \alpha \nabla_{\mathbf{w}_j} l(\mathbf{w}_j) \quad (\text{对每个 } j=1, 2, \dots, q)$$

$$\frac{\partial \mathbf{o}_j}{\partial \mathbf{w}_j} = \mathbf{x}$$

逻辑回归代价函数梯度计算推导

作业：

推导逻辑回归代价函数的梯度计算公式

推导**Softmax**回归代价函数的梯度计算公式

回归学习-Softmax回归

Softmax回归的梯度下降推导：

$$\begin{aligned}
 -\ell(\theta) &= -\sum_{i=1}^m \sum_{l=1}^k \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{\mathbf{1}(y^{(i)}=l)} \\
 &= -\sum_{i=1}^m \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \ln \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right) \\
 &= -\sum_{i=1}^m \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \left(\ln e^{\theta_l^T x^{(i)}} - \ln \sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \\
 &= -\sum_{i=1}^m \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \left(\theta_l^T x^{(i)} - \ln \sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \\
 &= -\sum_{i=1}^m \left(\sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \theta_l^T x^{(i)} - \sum_{l=1}^k \left(\mathbf{1}(y^{(i)}=l) \ln \sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \right) \\
 &= -\sum_{i=1}^m \left(\sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \theta_l^T x^{(i)} - \ln \left(\sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \right) \\
 &= -\sum_{i=1}^m \left(\sum_{l=1}^k \mathbf{1}(y^{(i)}=l) \theta_l^T x^{(i)} - \ln \left(\sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \right)
 \end{aligned}$$

Since $y^{(i)}$ will only take either one of the values in $1, 2, 3, \dots, k$

$$\therefore \sum_{l=1}^k \mathbf{1}(y^{(i)}=l) = \mathbf{1}(y^{(i)}=1) + \mathbf{1}(y^{(i)}=2) + \dots + \mathbf{1}(y^{(i)}=k) = 1$$

回归学习-Softmax回归

Softmax回归的梯度下降推导：

$$\begin{aligned}
 \frac{\partial}{\partial \theta_{gh}}(-\ell(\theta)) &= \frac{\partial}{\partial \theta_{gh}} \left(- \sum_{i=1}^m \left(\sum_{l=1}^k \mathbf{1}(y^{(i)} = l) \theta_l^T x^{(i)} - \ln \left(\sum_{j=1}^k e^{\theta_j^T x^{(i)}} \right) \right) \right) \quad \text{where } g = 1, 2, \dots, k \text{ and } h = 0, 1, 2, \dots, n \\
 &= - \sum_{i=1}^m \left(\mathbf{1}(y^{(i)} = g) x_h^{(i)} - \frac{1}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \cdot x_h^{(i)} e^{\theta_g^T x^{(i)}} \right) \\
 &= - \sum_{i=1}^m \left(\mathbf{1}(y^{(i)} = g) - \frac{e^{\theta_g^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right) x_h^{(i)} \\
 &= \sum_{i=1}^m \left(\frac{e^{\theta_g^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} - \mathbf{1}(y^{(i)} = g) \right) x_h^{(i)}
 \end{aligned}$$

$$\theta_{gh} := \theta_{gh} - \alpha \sum_{i=1}^m \left(\frac{e^{\theta_g^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} - \mathbf{1}(y^{(i)} = g) \right) x_h^{(i)} \quad \text{where } \alpha \text{ is the learning rate}$$